

NOTE:

The 1st transaction is called the one shown on the left in the screenshots, and the 2nd is called the one on the right.

- Create a table called employee with columns id serial, name varchar, status varchar.
- Replicate the example given in the lecture with the code:

### – 1st scenario

Two separate transactions are started. The insert occurred in the 1st transaction that is not visible in the 2nd transaction

The left screenshot (Script-8) shows the following SQL code:

```
CREATE TABLE public.employee (  
-- first transaction  
begin;  
select txid_current();  
insert into public.employee ("name", status)  
values ('Alice', 'Not fired');  
select *, xmin, xmax from public.employee e;  
commit;
```

The right screenshot (Script-9) shows the following SQL code:

```
-- first transaction  
begin;  
select *, xmin, xmax from public.employee;  
commit;  
-- second transaction  
begin;  
select txid_current();  
delete from public.employee  
where id = 1;  
select *, xmin, xmax  
from public.employee e;
```

Both screenshots show a table with columns: id, name, status, xmin, xmax. The left window shows a row with id 1, name Alice, status Not fired, xmin 980, and xmax 0. The right window shows an empty table.

After the committing 1st transaction, the inserted row is appeared in the second transaction - **Phantom Read anomaly** happened in the scope of 2nd transaction

**\*<postgres> Script-8**

```

CREATE TABLE public.employee (
-- first transaction
begin;
select txid_current();
insert into public.employee ("name", status)
values ('Alice', 'Not fired');
select *, xmin, xmax from public.employee e;
commit;

```

**Statistics 1**

| Name         | Value                         |
|--------------|-------------------------------|
| Updated Rows | 0                             |
| Execute time | 0.002s                        |
| Start time   | Wed Jul 08 22:18:41 EEST 2026 |
| Finish time  | Wed Jul 08 22:18:41 EEST 2026 |
| Query        | commit                        |

**\*<postgres> Script-9**

```

-- first transaction
begin;
select *, xmin, xmax from public.employee;
commit;

-- second transaction
begin;
select txid_current();
delete from public.employee
where id = 1;
select *, xmin, xmax
from public.employee e;

```

**employee 1**

| id | name  | status    | xmin | xmax |
|----|-------|-----------|------|------|
| 1  | Alice | Not fired | 980  | 0    |

## – 2nd scenario

The row was deleted in the scope of uncommitted 2nd transaction, so it is still visible on the SELECT command of 1st transaction and disappeared after the 2nd transaction commit

**Non-Repeatable Read anomaly** happened in the scope of 2nd transaction/ **'Read committed'** isolation level

**\*<postgres> Script-8**

```

-- first transaction
begin;
select txid_current();
insert into public.employee ("name", status)
values ('Alice', 'Not fired');
select *, xmin, xmax from public.employee e;
commit;

-- second transaction
begin;
select *, xmin, xmax
from public.employee;
commit;

```

**employee 1**

| id | name  | status    | xmin | xmax |
|----|-------|-----------|------|------|
| 1  | Alice | Not fired | 980  | 981  |

**\*<postgres> Script-9**

```

begin;
select *, xmin, xmax from public.employee;
commit;

-- second transaction
begin;
select txid_current();
delete from public.employee
where id = 1;
select *, xmin, xmax
from public.employee e;
commit;

```

**Statistics 1**

| Name         | Value                                    |
|--------------|------------------------------------------|
| Updated Rows | 1                                        |
| Execute time | 0.002s                                   |
| Start time   | Wed Jul 08 22:27:26 EEST 2026            |
| Finish time  | Wed Jul 08 22:27:26 EEST 2026            |
| Query        | delete from public.employee where id = 1 |

**\*<postgres> Script-8**

```

-- first transaction
begin;

select txid_current();

insert into public.employee ("name", status)
values ('Alice', 'Not fired');

select *, xmin, xmax from public.employee e;

commit;

-- second transaction
begin;

select *, xmin, xmax
from public.employee;

commit;

```

**employee 1**

select \*, xmin, xmax from public.employee

| id  | name  | status    | xmin | xmax |
|-----|-------|-----------|------|------|
| 123 | Alice | Not fired |      |      |

**\*<postgres> Script-9**

```

begin;

select *, xmin, xmax from public.employee e;

commit;

-- second transaction
begin;

select txid_current();

delete from public.employee
where id = 1;

select *, xmin, xmax
from public.employee e;

commit;

```

**Statistics 1**

| Name         | Value                         |
|--------------|-------------------------------|
| Updated Rows | 0                             |
| Execute time | 0.003s                        |
| Start time   | Wed Jul 08 22:29:16 EEST 2026 |
| Finish time  | Wed Jul 08 22:29:16 EEST 2026 |
| Query        | commit                        |

## – 3rd scenario

The 2nd transaction updates the table data change is not reflected in the scope of 1st transaction before commit but displayed after the commit.

**Non-Repeatable Read anomaly** happened/update scenario

**Script-8**

```

select *, xmin, xmax
from public.employee;

commit;

insert into public.employee ("name", status)
values ('Alice', 'Not fired');

-- third transaction
begin;

select *, xmin, xmax
from public.employee e;

commit;

```

**employee 1**

select \*, xmin, xmax from public.employee

| id | name  | status    | xmin | xmax |
|----|-------|-----------|------|------|
| 2  | Alice | Not fired | 982  | 983  |

**Script-9**

```

delete from public.employee
where id = 1;

select *, xmin, xmax
from public.employee e;

commit;

-- third transaction
begin;

select txid_current();

update public.employee
set status = 'Fired'
where id = 2;

select *, xmin, xmax
from public.employee e;

commit;

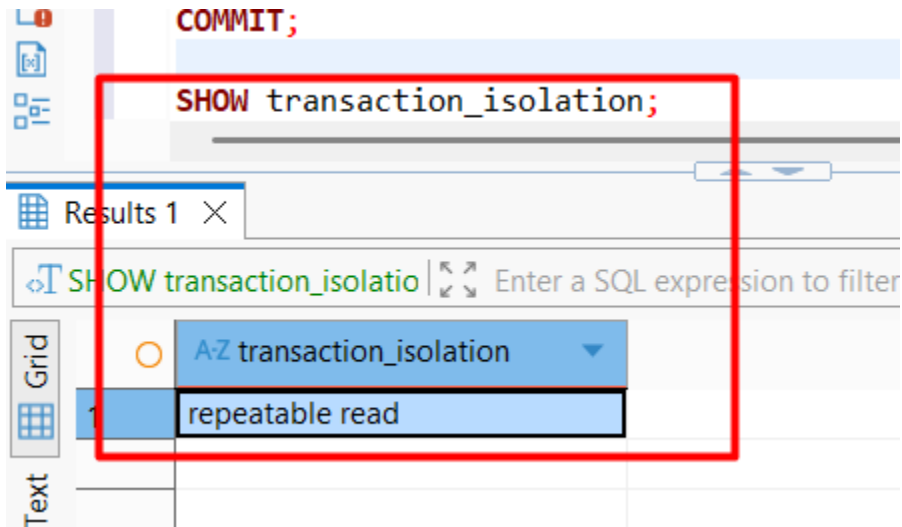
```

**employee 1**

select \*, xmin, xmax from public.employee

| id | name  | status | xmin | xmax |
|----|-------|--------|------|------|
| 2  | Alice | Fired  | 983  | 0    |

–Run the command set transaction isolation level repeatable read.

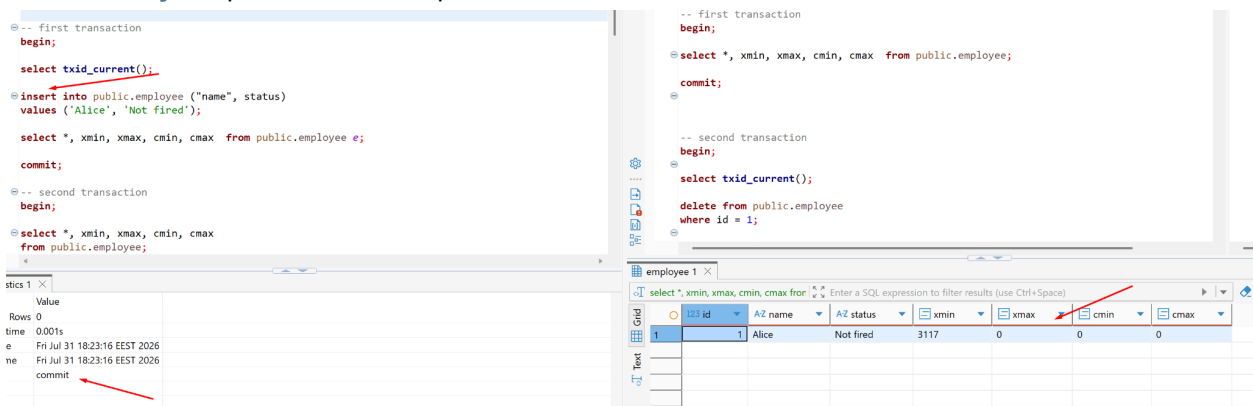


–Check your current isolation level in each session with show transaction isolation level.

## – 1st scenario

Two separate transactions are started. The insert occurred in the 1st transaction that is not visible in the 2nd transaction

After the committing 1st transaction, the inserted row is appeared in the second transaction - **phantom read anomaly** still present in the scope of 2nd transaction



## – 2nd scenario

The row was deleted in the scope of uncommitted 2nd transaction, so it is still visible on the SELECT command of 1st transaction and **NOT** disappeared after the 2nd transaction commit  
**Non-Repeatable Read anomaly** disappeared

```

commit;

-- second transaction
begin;

select *, xmin, xmax, cmin, cmax
from public.employee;

commit;

insert into public.employee ("name", status)

```

```

-- second transaction
begin;

select txid_current();

delete from public.employee
where id = 1;

select *, xmin, xmax, cmin, cmax
from public.employee;

```

```

select *, xmin, xmax, cmin, cmax from public.employee e;

commit;

-- second transaction
begin;

select *, xmin, xmax, cmin, cmax
from public.employee;

commit;

insert into public.employee ("name", status)

```

```

-- second transaction
begin;

select txid_current();

delete from public.employee
where id = 1;

select *, xmin, xmax, cmin, cmax
from public.employee;

commit;
rollback

```

| id | name | status    | xmin | xmax | cmin | cmax |
|----|------|-----------|------|------|------|------|
| 1  | Bob  | Not fired | 3119 | 0    | 0    | 0    |

| Name         | Value                                    |
|--------------|------------------------------------------|
| Updated Rows | 0                                        |
| Execute time | 0.001s                                   |
| Start time   | Fri Jul 31 18:34:55 EEST 2026            |
| Finish time  | Fri Jul 31 18:34:55 EEST 2026            |
| Query        | delete from public.employee where id = 1 |

| id | name | status    | xmin | xmax | cmin | cmax |
|----|------|-----------|------|------|------|------|
| 1  | Bob  | Not fired | 3119 | 0    | 0    | 0    |

| Name         | Value                         |
|--------------|-------------------------------|
| Updated Rows | 0                             |
| Execute time | 0.001s                        |
| Start time   | Fri Jul 31 18:36:04 EEST 2026 |
| Finish time  | Fri Jul 31 18:36:04 EEST 2026 |
| Query        | commit                        |

## – 3rd scenario

The 2nd transaction updates the table data change is not reflected in the scope of 1st transaction before and **after** commit (1st is still uncommitted). **Non-Repeatable Read anomaly** disappeared

```

-- third transaction
begin;

select *, xmin, xmax, cmin, cmax
from public.employee;

commit;

```

```

-- third transaction
begin;

select txid_current();

update public.employee
set status = 'Fired'
where id = 2;

select *, xmin, xmax, cmin, cmax
from public.employee;

commit;

```

| id | name | status    | xmin | xmax | cmin | cmax |
|----|------|-----------|------|------|------|------|
| 2  | Bob  | Not fired | 3119 | 3120 | 0    | 0    |

| Name         | Value                         |
|--------------|-------------------------------|
| Updated Rows | 0                             |
| Execute time | 0.001s                        |
| Start time   | Fri Jul 31 18:40:17 EEST 2026 |
| Finish time  | Fri Jul 31 18:40:17 EEST 2026 |
| Query        | commit                        |

–Recreate employee table and redo the second task but modify the code so that select statements would now include cmin and cmax system columns. What changed?

Nothing changed when the tasks flow repeated. Because we had one query per transaction. To see these columns change, a few inserts within 1 transaction were performed.

|   | AZ name | AZ status | xmin | xmax | cmin | cmax |
|---|---------|-----------|------|------|------|------|
| 2 | Alice   | Fired     | 1015 | 0    | 0    | 0    |
| 3 | Bob     | Not fired | 1015 | 0    | 1    | 1    |
| 4 | Carol   | Not fired | 1015 | 0    | 2    | 2    |

-Try to cause a serialization anomaly on the employee table (add more data if necessary). Change your isolation level to serializable and try to cause serialization anomaly one more time. What happened?

## Serialization anomaly demonstration

Update inside 1st transaction is not visible on 2nd

```
--Serialization anomaly
BEGIN;
SELECT count(*) FROM employee WHERE status='Not fired';
UPDATE employee SET status='Fired' WHERE id=1;
COMMIT;
SELECT * FROM employee;
```

```
--Serialization anomaly
BEGIN;
SELECT count(*) FROM employee WHERE status='Not fired';
UPDATE employee SET status='Fired' WHERE id=2;
COMMIT;
SELECT * FROM employee;
```

| id | name  | status    |
|----|-------|-----------|
| 1  | Alice | Fired     |
| 2  | Bob   | Not fired |

| id | name  | status    |
|----|-------|-----------|
| 1  | Alice | Not fired |
| 2  | Bob   | Not fired |

And vice versa

```
--Serialization anomaly
BEGIN;
SELECT count(*) FROM employee WHERE status='Not fired';
UPDATE employee SET status='Fired' WHERE id=1;
COMMIT;
SELECT * FROM employee;
```

```
--Serialization anomaly
BEGIN;
SELECT count(*) FROM employee WHERE status='Not fired';
UPDATE employee SET status='Fired' WHERE id=2;
COMMIT;
SELECT * FROM employee;
```

| id | name  | status    |
|----|-------|-----------|
| 1  | Alice | Fired     |
| 2  | Bob   | Not fired |

| id | name  | status    |
|----|-------|-----------|
| 1  | Alice | Not fired |
| 2  | Bob   | Fired     |

After commits both updates are done(fired)

```
--Serialization anomaly
BEGIN;
SELECT count(*) FROM employee WHERE status='Not fired';
UPDATE employee SET status='Fired' WHERE id=1;
COMMIT;
SELECT * FROM employee;
```

```
--Serialization anomaly
BEGIN;
SELECT count(*) FROM employee WHERE status='Not fired';
UPDATE employee SET status='Fired' WHERE id=2;
COMMIT;
SELECT * FROM employee;
```

| id | name  | status |
|----|-------|--------|
| 1  | Alice | Fired  |
| 2  | Bob   | Fired  |

After setting isolation level to SERIALIZABLE, the error is thrown on serialization anomaly

```
--Serialization anomaly
BEGIN;
SELECT count(*) FROM employee WHERE status='Not fired';
UPDATE employee SET status='Fired' WHERE id=1;
COMMIT;
SELECT * FROM employee;

BEGIN TRANSACTION ISOLATION LEVEL SERIALIZABLE;
SELECT * FROM employee;
UPDATE employee SET status = 'Fired' WHERE id = 1;
COMMIT;
```

```
--Serialization anomaly
BEGIN;
SELECT count(*) FROM employee WHERE status='Not fired';
UPDATE employee SET status='Fired' WHERE id=2;
COMMIT;
SELECT * FROM employee;

BEGIN TRANSACTION ISOLATION LEVEL SERIALIZABLE;
SELECT * FROM employee;
UPDATE employee SET status = 'Fired' WHERE id = 2;
COMMIT;

begin;
```

SQL Error [40001]: ERROR: could not serialize access due to read/write dependencies among transactions  
 Detail: Reason code: Canceled on identification as a pivot, during commit attempt.  
 Hint: The transaction might succeed if retried.  
 Error position:

## --LOST UPDATE ANOMALY

The 1st transaction updates the salary and commits. Parallel 2nd transaction updates salary also.

```
-- LOST ANOMALY
CREATE TABLE public.employee (id serial, name varchar, salary int);
INSERT INTO employee (name, salary) VALUES ('Alice', 1000);

BEGIN;
SELECT salary FROM employee WHERE id=1;
UPDATE employee SET salary = 1000 + 100 WHERE id=1;
commit;
```

```
-- LOST ANOMALY
BEGIN;
SELECT salary FROM employee WHERE id=1;
UPDATE employee SET salary = 1000 + 500 WHERE id=1;
commit;

SET SESSION CHARACTERISTICS AS TRANSACTION ISOLATION LEVEL Serializable;
```

| id | salary |
|----|--------|
| 1  | 1,500  |

So select after 2nd transaction commit shows 'wrong' salary - update is lost

```
--LOST ANOMALY
CREATE TABLE public.employee (id serial, name varchar, salary int);
INSERT INTO employee (name, salary) VALUES ('Alice', 1000);

BEGIN;
SELECT salary FROM employee WHERE id=1;
UPDATE employee SET salary = 1000 + 100 WHERE id=1;
commit;
```

|                             |                                                           |
|-----------------------------|-----------------------------------------------------------|
| employee 1                  |                                                           |
| SELECT salary FROM employee | Enter a SQL expression to filter results (use Ctrl+Space) |
| 123 salary                  |                                                           |
| 1                           | 1,500                                                     |

```
update public.employee set status = 'Fired' where name = 'Carol';
commit;
```

--LOST ANOMALY

```
BEGIN;
SELECT salary FROM employee WHERE id=1;
UPDATE employee SET salary = 1000 + 500 WHERE id=1;
commit;
```

|                                   |                                                           |
|-----------------------------------|-----------------------------------------------------------|
| employee 1                        |                                                           |
| SELECT salary FROM employee WHERE | Enter a SQL expression to filter results (use Ctrl+Space) |
| 123 salary                        |                                                           |
| 1                                 | 1,500                                                     |